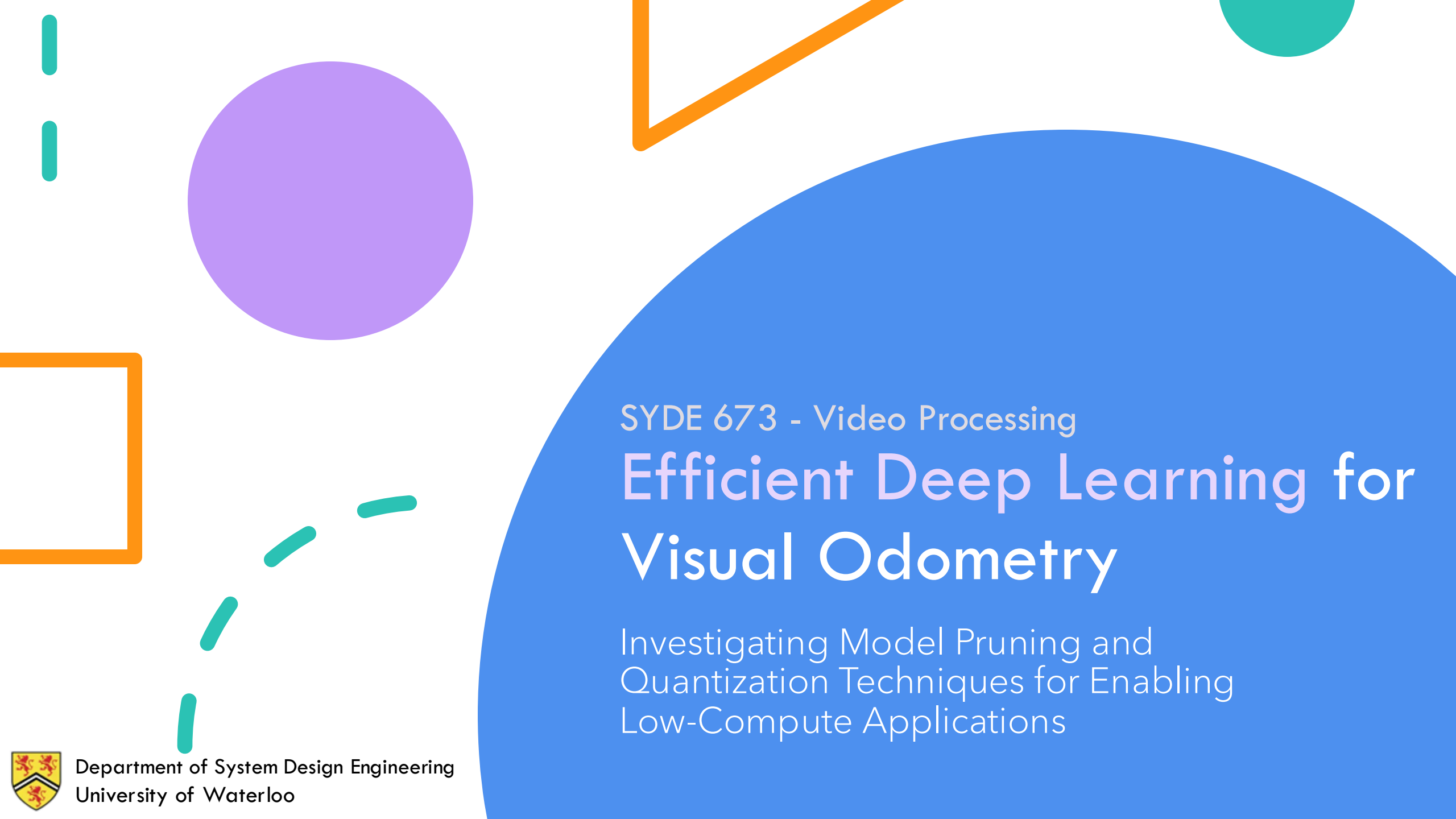




SYDE 673 - Video Processing

Efficient Deep Learning for Visual Odometry

Investigating Model Pruning and
Quantization Techniques for Enabling
Low-Compute Applications



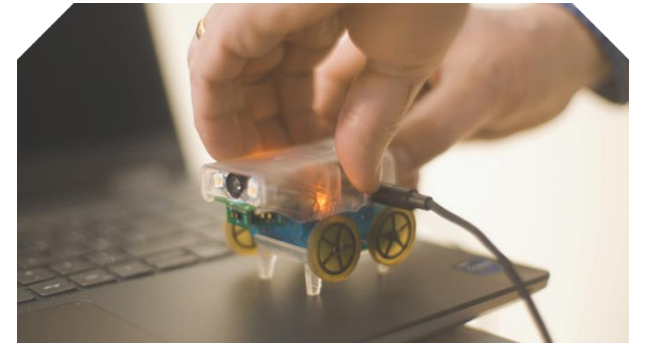
Motivation

Visual Odometry for Resource-Constrained Systems

Autonomous systems in low-compute environments demand efficient and accurate visual odometry. However, there is a **fundamental trade-off** between:

- **Model Accuracy**: Reliable navigation and mapping
- **Computational Efficiency**: Real-time performance and low power consumption

Resource-constrained autonomous systems include **drones for surveillance/mapping/delivery** and **robots for search/rescue/monitoring**.



Exploring the trade-off between model accuracy and computational efficiency

Objectives:

1. To demonstrate the accuracy of transformer-based visual odometry (VO) compared to classical methods
2. To optimize various components of the VO process with respect to computational efficiency using strategies such as quantization and model pruning

Visual Odometry: Classical Approaches

Visual Odometry estimates a camera's position, orientation, and motion by analyzing changes between consecutive image frames.

Feature-based methods estimate motion using distinct features, offering high accuracy and robustness. However, they struggle with repetitive patterns, blurry images, and lighting/viewpoint changes.

Appearance-based methods estimate motion by analyzing intensity patterns, offering efficiency and simplicity. However, they're sensitive to lighting, viewpoint, and texture-less regions, requiring careful tuning.

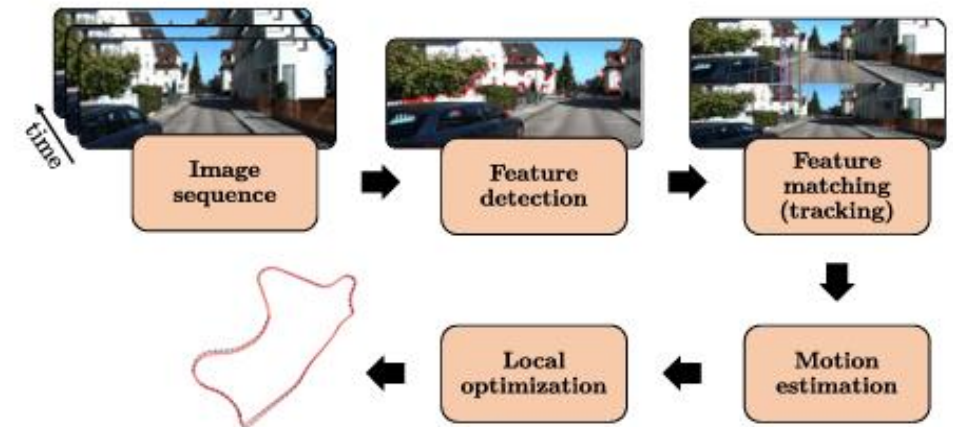
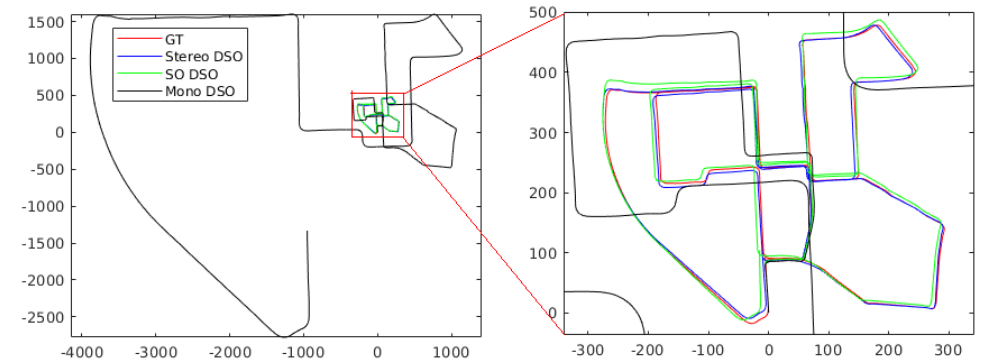
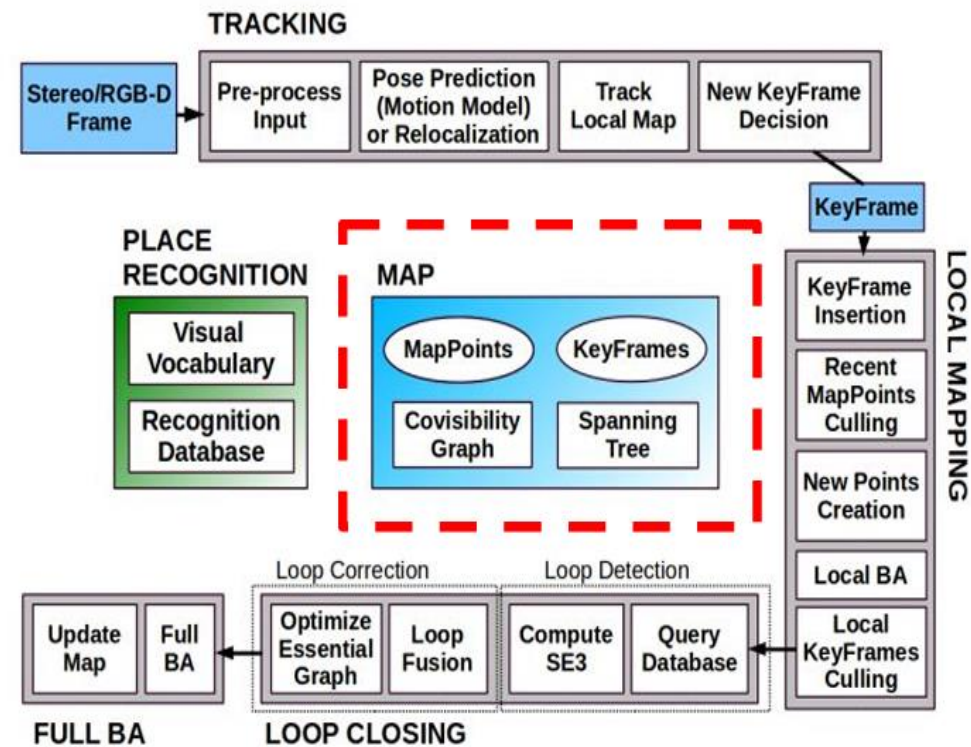
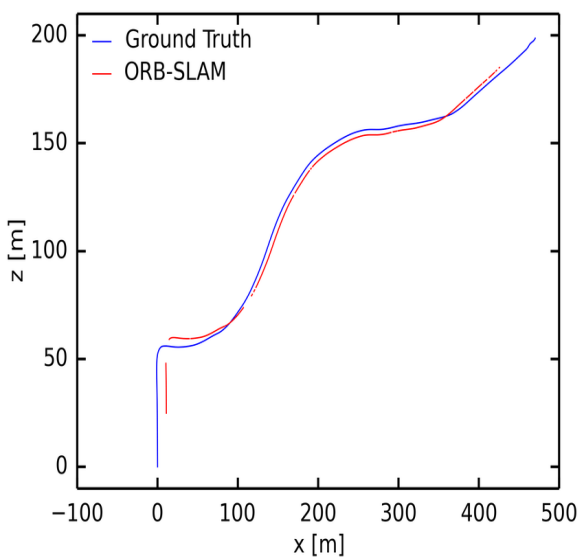
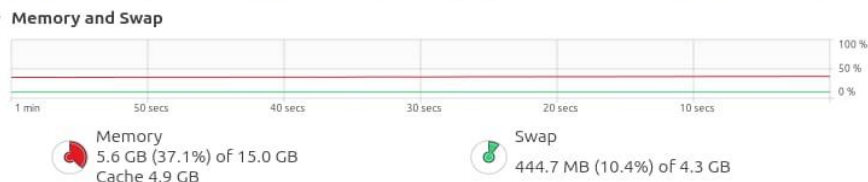
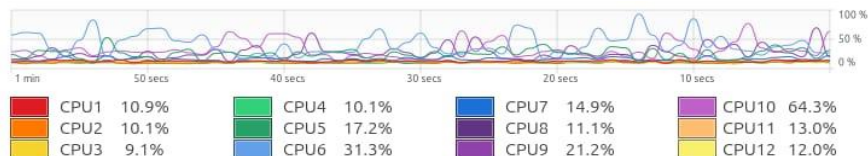


FIGURE 1. Traditional pipeline for visual odometry. The scenario images are taken from the KITTI odometry dataset [16].



ORB-Slam3

ORB Slam Results



Visual Odometry: Deep-Learning Approaches

How Deep-Learning Approaches Work

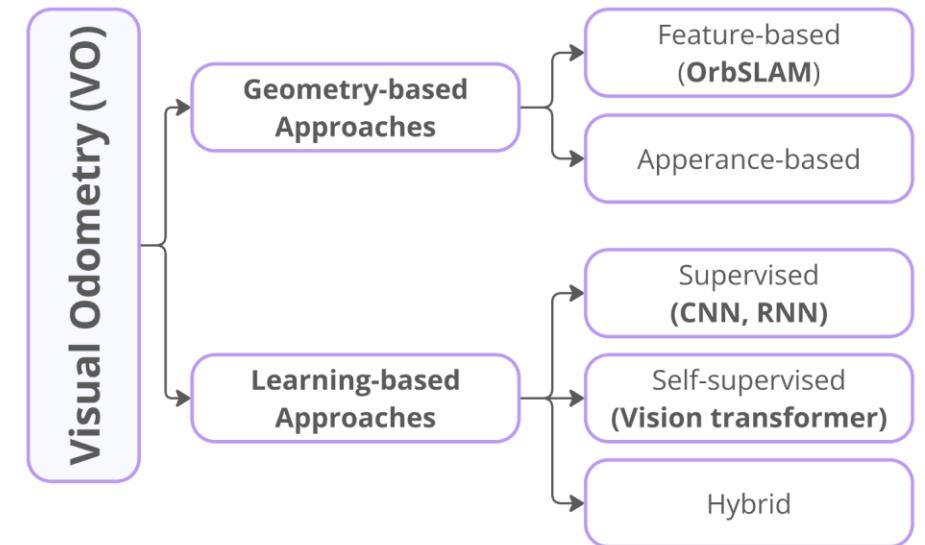
- Use convolutional neural networks (CNNs) to **extract features** from images
- Employ recurrent neural networks (RNNs) or other **sequential models** to **estimate motion** from feature sequences
- Often use photometric or geometric error metrics for training

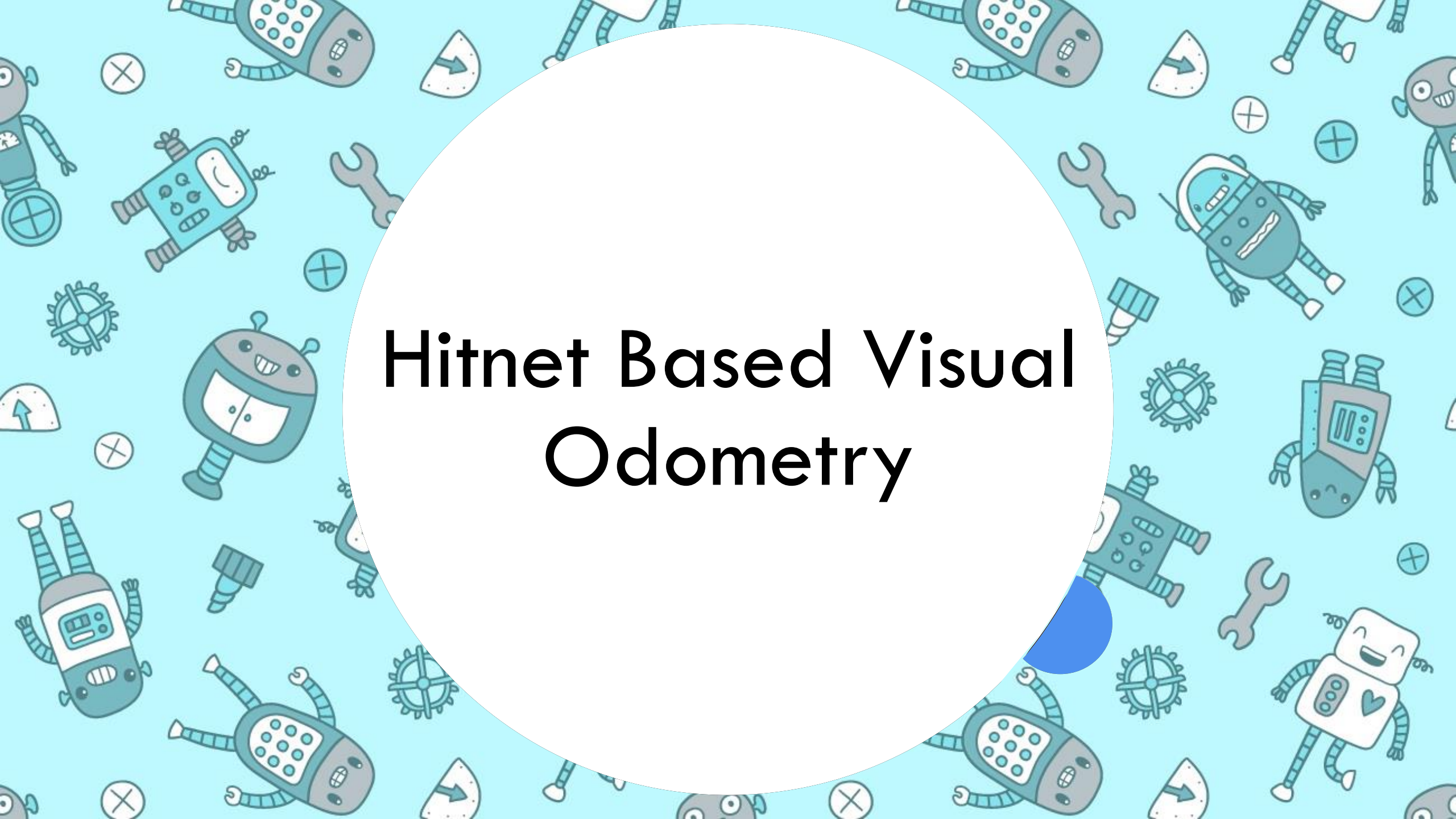
Advantages

- Can learn robust features and motion patterns from data
- Can handle complex scenes and lighting conditions
- Can be optimized for specific platforms or applications

Limitations

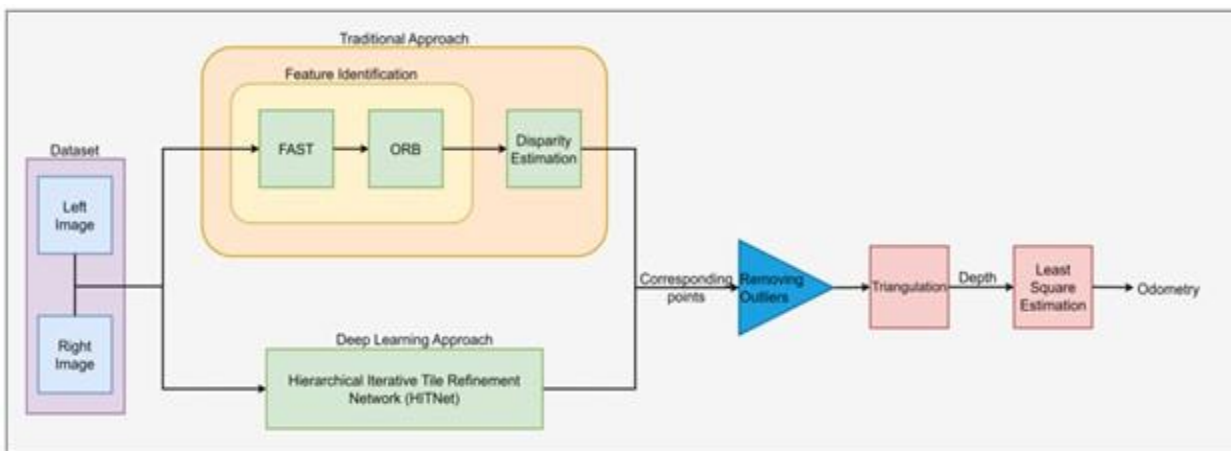
- Require large amounts of labeled training data
- **Can be computationally expensive and memory-intensive**
- May not generalize well to new or unseen environments





Hitnet Based Visual Odometry

Hitnet Based Visual Odometry



```

=====
Frame: 1200
Estimated Position (HITNET): [509.8940635 -36.51326244 30.48466327]
Ground Truth Position: [545.2426 -15.53084 -11.04965]
Angle Error - 2D: 10455.043362997172, 3D: 9881.685663750193, Hitnet: 15202.080981790972
RMSE - 2D: 91564.68812759842, 3D: 48143.7570484415, Hitnet: 63863.19677531179
=====
  
```

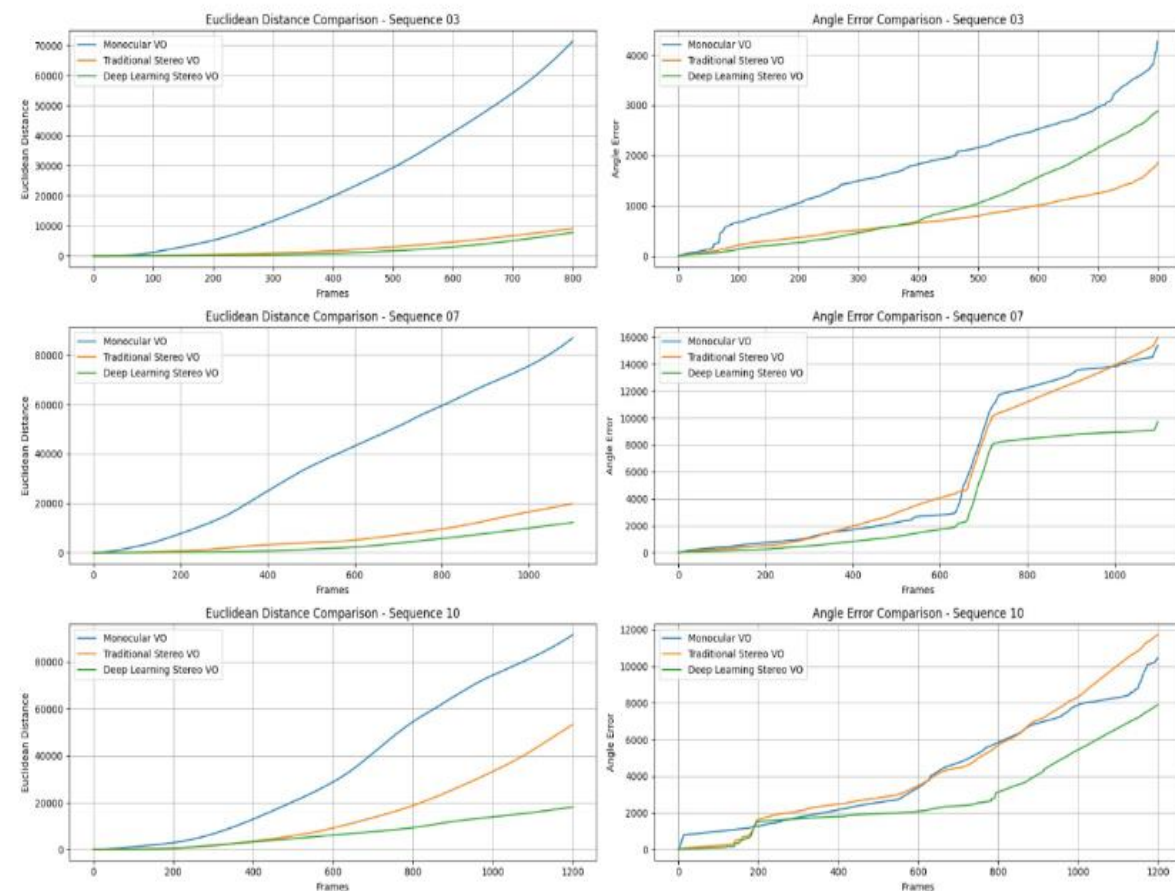


Figure 2: Error Plots

[1] V. Tankovich et al., "HITNET: Hierarchical Iterative Tile Refinement Network for real-time stereo matching," 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Jun. 2022, doi: 10.1109/cvpr46437.2021.01413.

Visual Odometry: Vision Transformer

How Vision Transformers Work

- Divide images into patches (tokens)
- Linearly embed and add positional encoding
- Apply transformer encoder with **self-attention** and FFNNs

Advantages

- Parallelization and scalability
- **Capture global context and long-range dependencies**
- Flexibility and **adaptability** to various vision tasks
- **Robustness to image distortions** and adversarial attacks

Limitations

- **High computational cost**
- Require large labeled training datasets
- Difficulty in interpretability and understanding

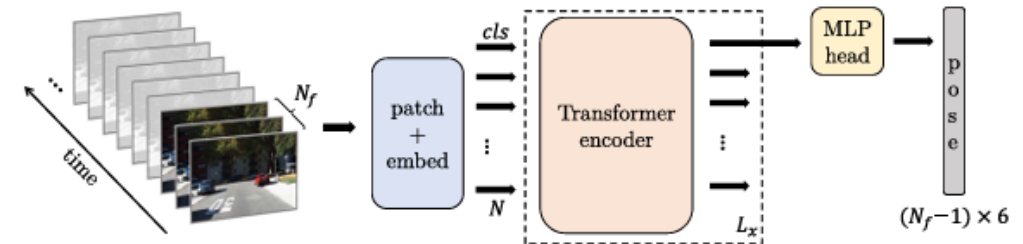


FIGURE 2. TSformer-VO pipeline. The input clips with N_f frames are processed into N patches. Each patch is embedded into tokens and sent to the sequence of Transformer blocks. A special vector called class token (cls) gathers the information from all patches and passes through the final MLP head, outputting the 6-DoF for the $N_f - 1$ estimated poses.

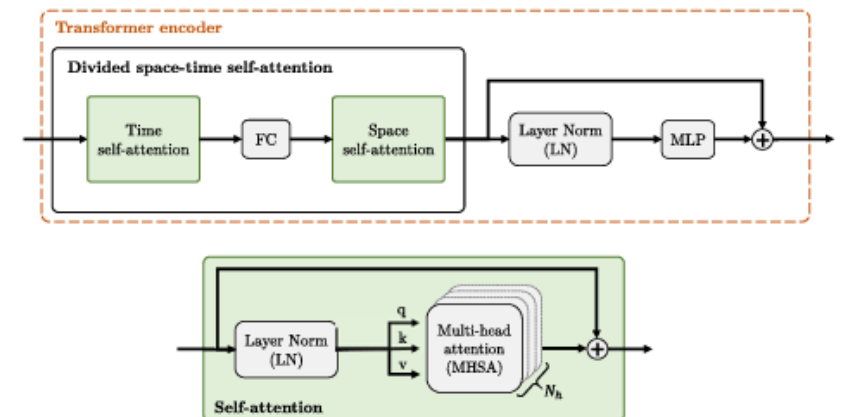


FIGURE 3. Transformer encoder with the divided space-time self-attention architecture. The illustration of the encoder was inspired by [14].

Visual Odometry: Research Gap

Method	Advantages	Disadvantages
Classical Methods	Fast, light-weight	Sensitive to lighting
CNNs / RNNs	Robust to Lighting, Handles Complex Scenes	Computationally Expensive
Vision Transformer	Parallelizable, Captures Global Context	Computationally Expensive

Optimization Approaches for Vision Transformers

1

Refining the
Pose
Estimation
Loss Function

2

Quantization

3

Pruning

TSformer-VO_[2]

Idea

End-to-end transformer model that treats monocular VO as a video understanding task, using attention to model spatio-temporal relationships across frames.

Approach

- **Divided Space-Time Attention** - Efficiently captures motion by processing spatial and temporal dimensions separately
- **Clip-to-Pose Regression** - Takes N-frame clips as input, outputs all 6-DoF poses at once
- **Sliding Window Inference** - Uses overlapping clips with averaging for smooth trajectories

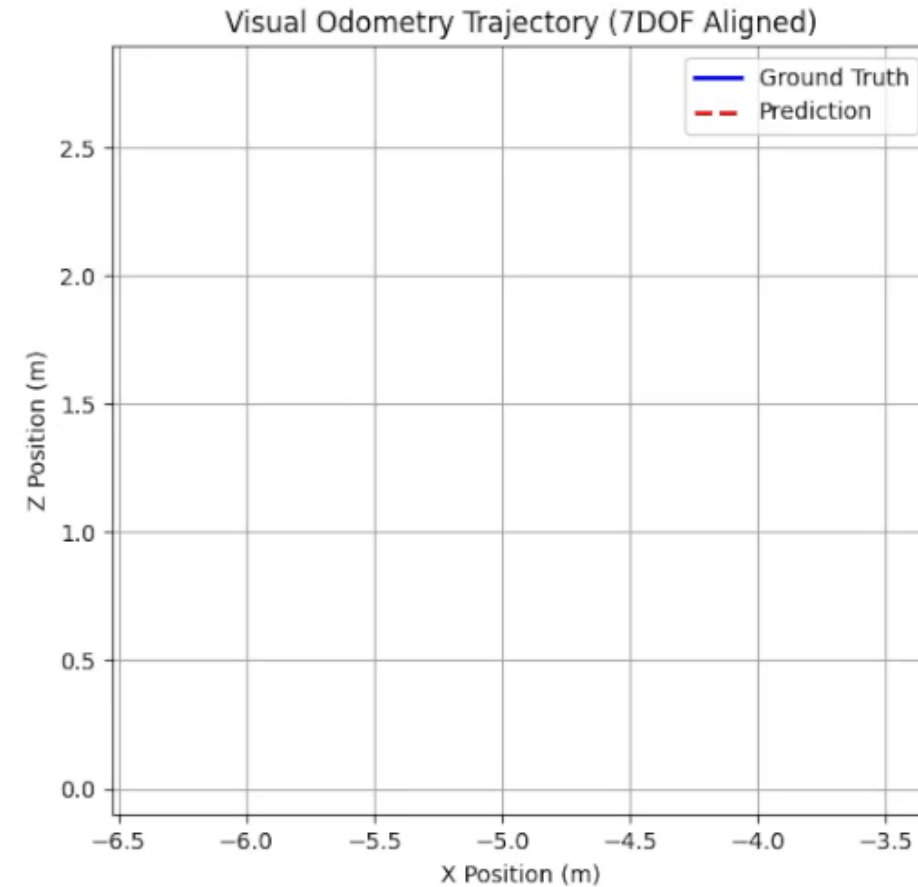
Metric	TSformer-VO	DeepVO	ORB-SLAM3
ATE (m)	3.21	4.12	2.98
RPE (m)	0.012	0.018	0.010
Inference time	20-40 ms	50 ms	30 ms

```

VisionTransformer(
  (dropout): Dropout(p=0.0, inplace=False)
  (patch_embed): PatchEmbed(
    (proj): Conv2d(3, 384, kernel_size=(16, 16), stride=(16, 16))
  )
  (pos_drop): Dropout(p=0.0, inplace=False)
  (time_drop): Dropout(p=0.0, inplace=False)
  (blocks): ModuleList(
    (0): Block(
      (norm1): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=384, out_features=1152, bias=True)
        (proj): Linear(in_features=384, out_features=384, bias=True)
        (proj_drop): Dropout(p=0.0, inplace=False)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (temporal_norm1): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
      (temporal_attn): Attention(
        (qkv): Linear(in_features=384, out_features=1152, bias=True)
        (proj): Linear(in_features=384, out_features=384, bias=True)
        (proj_drop): Dropout(p=0.0, inplace=False)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (temporal_fc): Linear(in_features=384, out_features=384, bias=True)
      (drop_path): Identity()
      (norm2): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
      (mlp): Mlp(
        (fc1): Linear(in_features=384, out_features=1536, bias=True)
        (act): GELU(approximate='none')
        (fc2): Linear(in_features=1536, out_features=384, bias=True)
        (drop): Dropout(p=0.0, inplace=False)
      )
    )
    (1-11): 11 x Block(
      (norm1): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=384, out_features=1152, bias=True)
        (proj): Linear(in_features=384, out_features=384, bias=True)
        (proj_drop): Dropout(p=0.0, inplace=False)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (temporal_norm1): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
      (temporal_attn): Attention(
        (qkv): Linear(in_features=384, out_features=1152, bias=True)
        (proj): Linear(in_features=384, out_features=384, bias=True)
        (proj_drop): Dropout(p=0.0, inplace=False)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (temporal_fc): Linear(in_features=384, out_features=384, bias=True)
      (drop_path): DropPath()
      (norm2): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
      (mlp): Mlp(
        (fc1): Linear(in_features=384, out_features=1536, bias=True)
        (act): GELU(approximate='none')
        (fc2): Linear(in_features=1536, out_features=384, bias=True)
        (drop): Dropout(p=0.0, inplace=False)
      )
    )
  )
  (norm): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
  (head): Linear(in_features=384, out_features=12, bias=True)
)

```

Qualitative evaluation and demonstration





Error Metrics

Metric	Description	What It Measures
Translational error (%)	% of drift in predicted vs ground truth position	Translation accuracy
Rotational error (deg/100m)	How much rotation error accumulates per 100 meters	Rotation alignment
ATE (Absolute Trajectory Error)	Absolute difference between predicted & ground truth trajectory	Global trajectory drift
RPE (Relative Pose Error - m)	Local translation error between consecutive frames	Short-term translation accuracy
RPE (Relative Pose Error - deg)	Local rotation error between consecutive frames	Short-term rotation consistency



Optimization Approach 1:

Refining the Pose Estimation Loss Function

Old Loss Function –

$\text{loss} = k * \text{MSE}(\text{estimated_angles}, \text{gt_angles}) + \text{MSE}(\text{estimated_translation}, \text{gt_translation})$

- Treats both rotation and translation as plain numbers.
 - Good for general regression, but MSE doesn't understand angular structure—so a 180° vs -180° could give a high error even though the orientation is the same.
- 



Optimization Approach 1:

Refining the Pose Estimation Loss Function

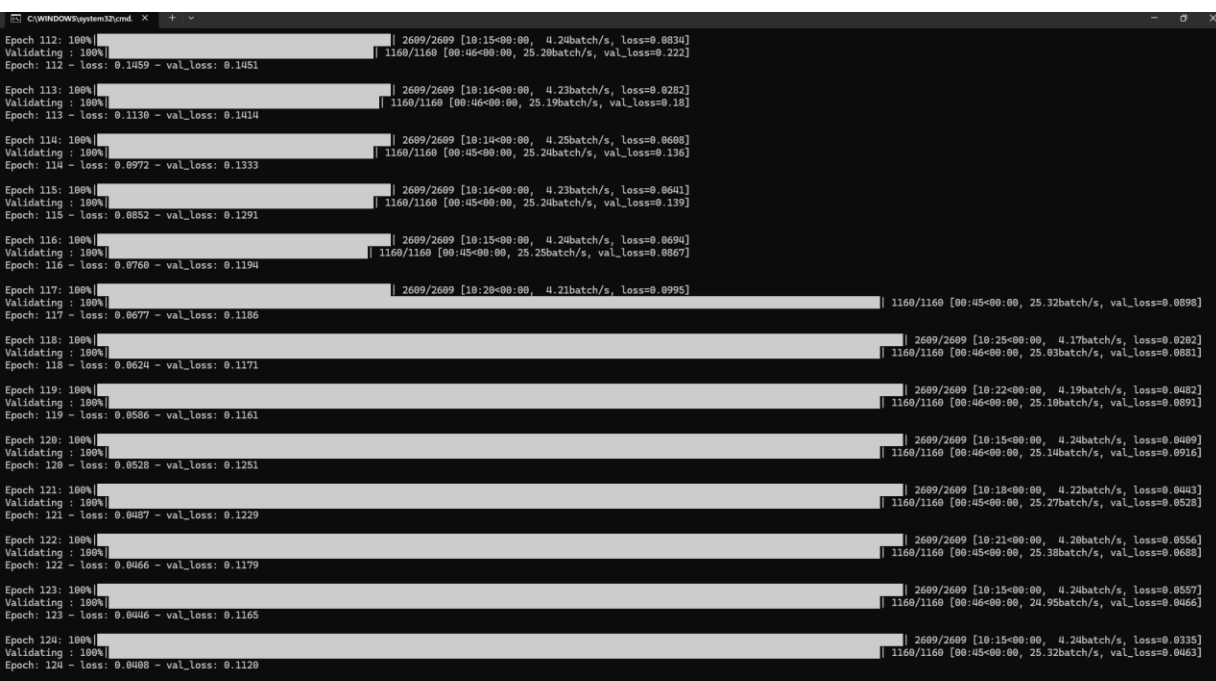
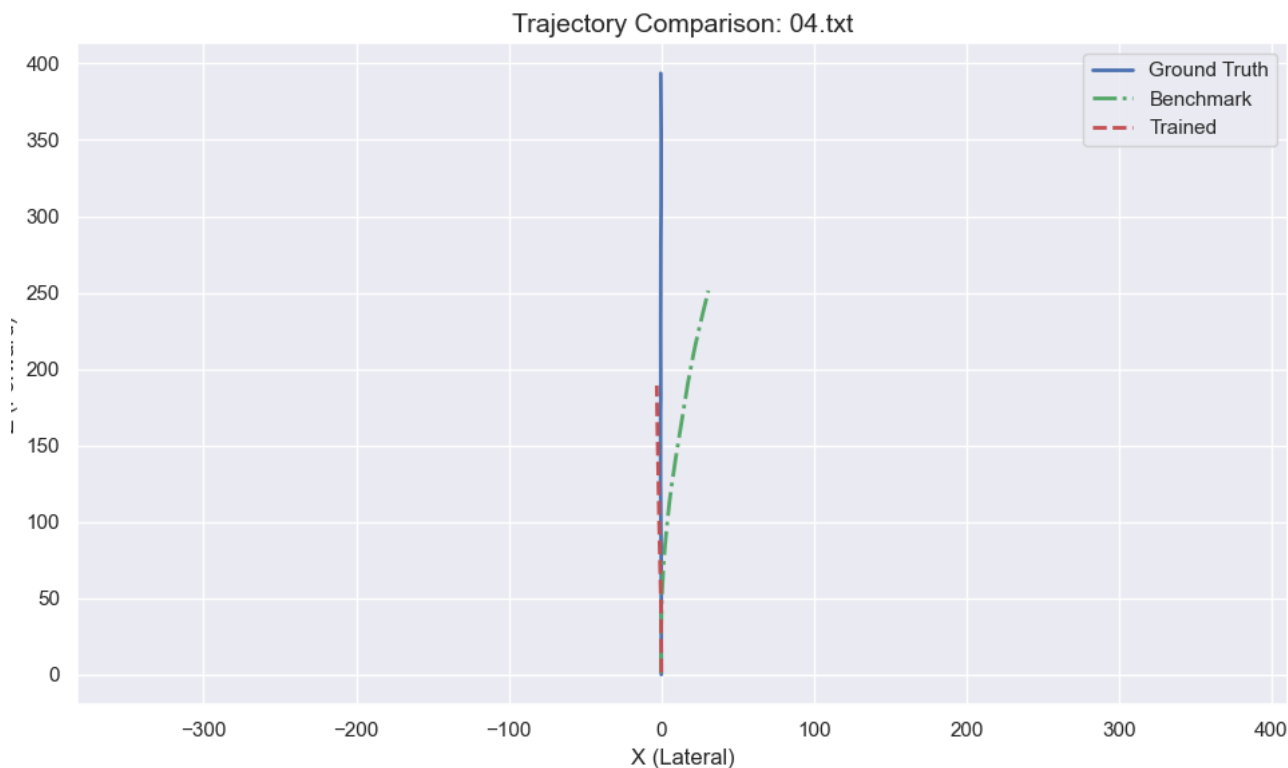
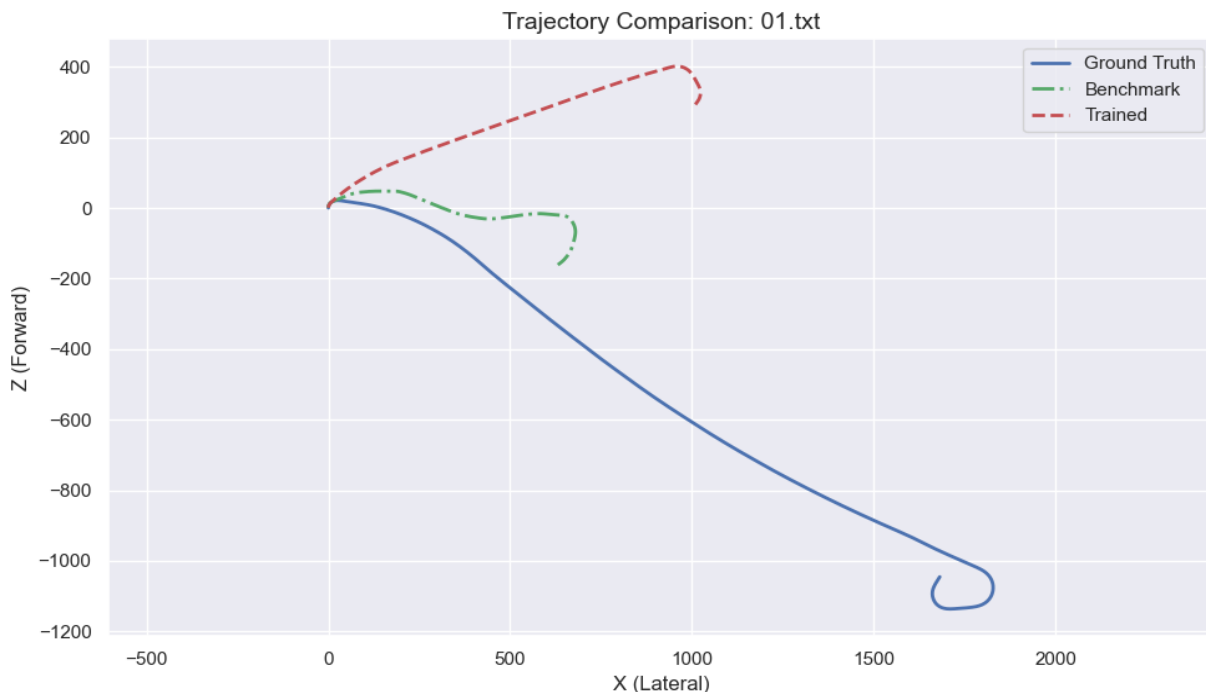
Updated Loss Function –

$\text{rotation_loss} = \text{mean}(1 - \cos(\text{pred_angles}, \text{gt_angles}))$

$\text{translation_loss} = \text{MSE}(\text{pred_translation}, \text{gt_translation})$

$\text{total_loss} = \lambda * \text{rotation_loss} + \text{translation_loss}$

- Rotation loss now uses cosine similarity, which measures the directional alignment of the angle vectors.
 - This is better for rotations because:
 - It captures the angular relationship.
 - Reduces penalization for mathematically "far but functionally similar" angles.
 - Translation loss still uses MSE, which is appropriate for linear distance changes.
- 



- The new loss function (Cosine + MSE) improves rotation accuracy, showing lower rotational error and RPE (deg) in both **Sequence 1** and **Sequence 4**.
- It also achieves better or comparable translation performance—especially in Sequence 4, where ATE and RPE (m) are significantly lower.
- Overall, it leads to more consistent and accurate pose predictions, particularly in orientation.

Metric	Seq 1 Benchmark	Seq 1 Our Model	Seq 4 Benchmark	Seq 4 Our Model
Translational error (%)	23.67	26.89	9.04	3.98
Rotational error (°/100m)	5.86	7.57	4.87	1.63
Absolute Trajectory Error (m)	101.70	68.34	6.00	2.22
RPE (m)	0.5415	0.4873	0.1414	0.1033
RPE (°)	0.3104	0.4022	0.1743	0.1627

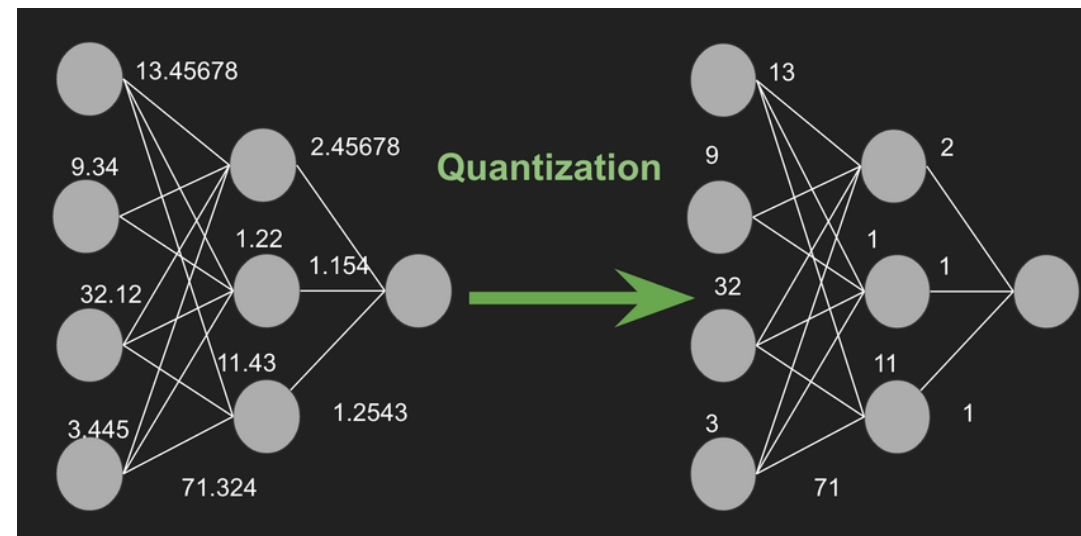
- The new loss function gives Better Directional awareness for pose.

Optimization Approach 2: Quantization

- Compared to the benchmark, the quantized models (Float16, Float32, and Float64) generally show a trade-off between **translation/rotation accuracy** and **ATE/RPE performance**.
- While the quantized versions exhibit **higher translational and rotational errors** than the benchmark—indicating a slight drop in pose estimation accuracy—they **significantly improve the Absolute Trajectory Error (ATE)** and **Relative Pose Error (RPE)** in both meters and degrees.
- This suggests that although the quantized models may deviate more in orientation and position globally, they maintain **better short-term consistency and smoother local motion**, likely due to numerical effects introduced by precision changes.

qint8	float16	float32	float64
Checkpoint Size	Checkpoint Size	Checkpoint Size	Checkpoint Size
32.52 MB	61.41 MB	122.73 MB	245.37 MB

- Quantized model's file size scaled linearly with data type

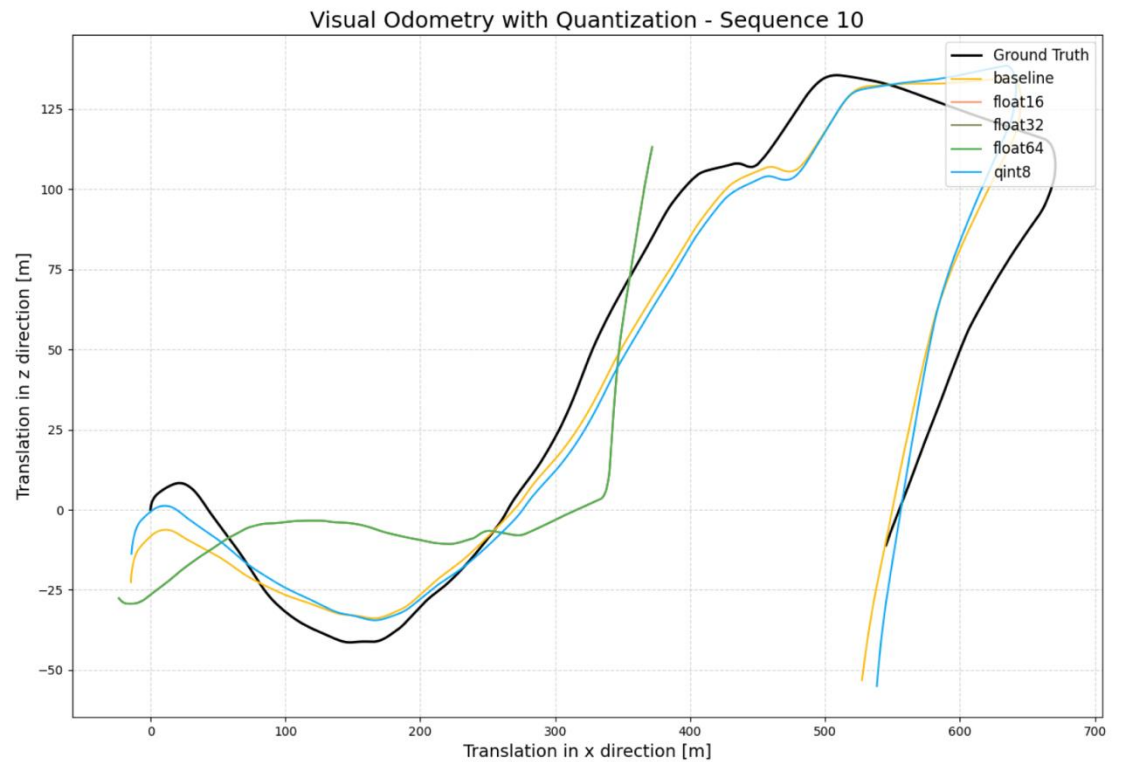
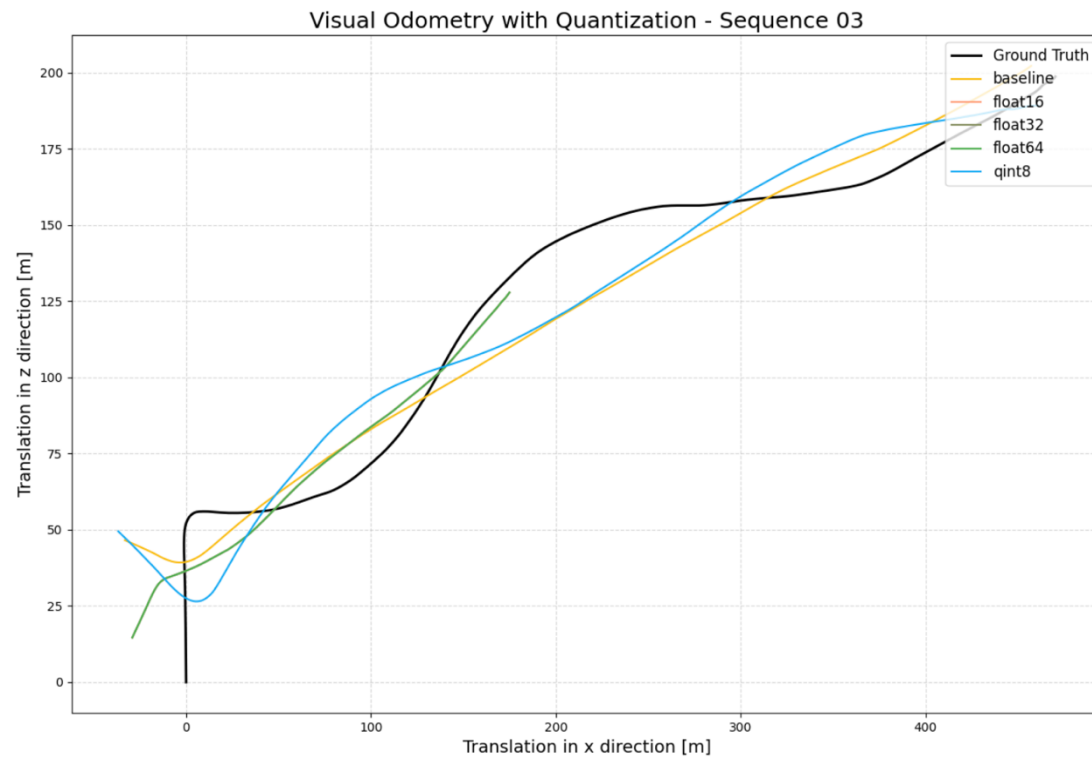


Quantized models surprisingly performed comparably to the baseline

Model	Translational error	Rotational error (deg/100m)	ATE (m)	RPE (m)	RPE (deg)
Benchmark	23.6%	5.86	101.70	0.541	0.310
Float64	30.4%	8.18	54.35	0.471	0.418
Float32	30.40%	8.27	54.35	0.471	0.418
Float16	31.41%	8.32	54.35	0.471	0.418

Quantization

Quantized models surprisingly performed comparably to the baseline

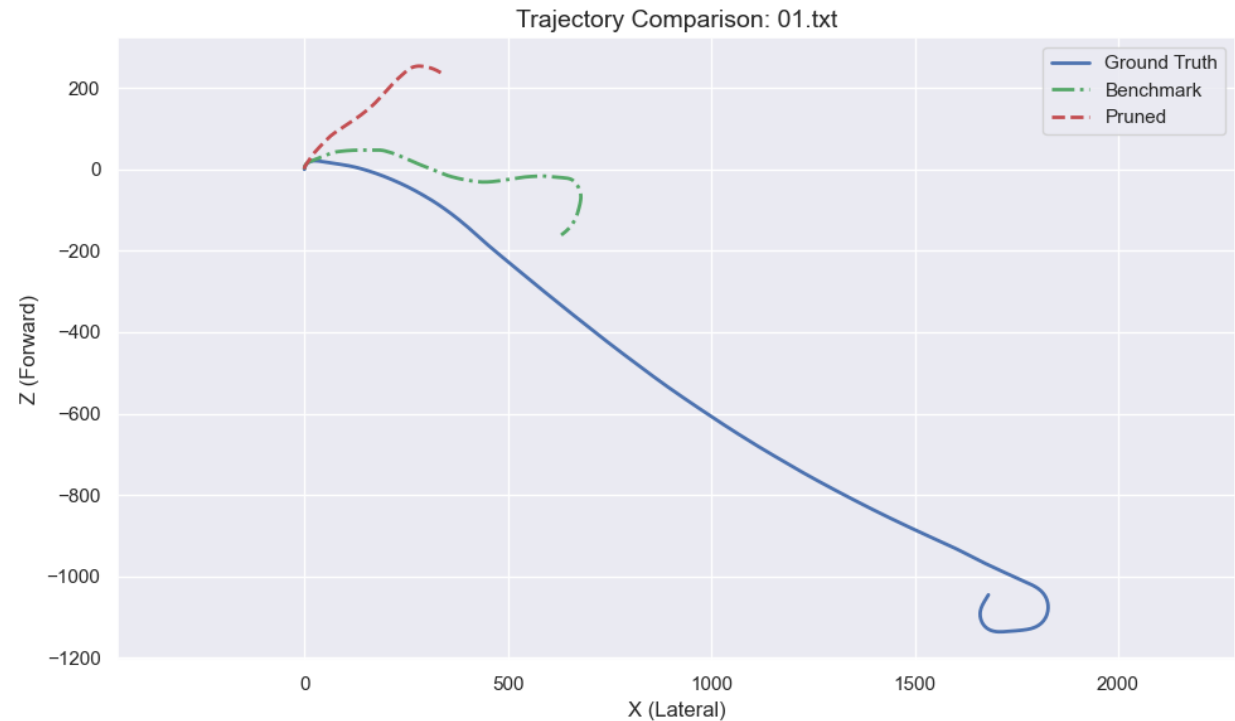
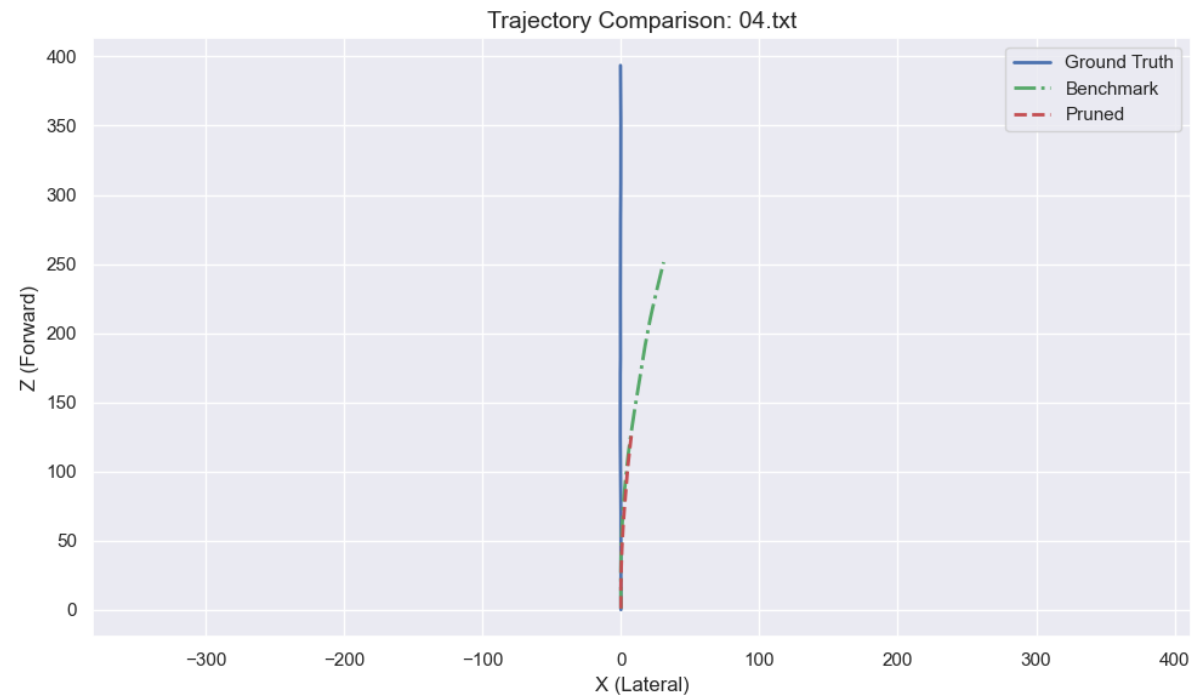


Optimization Approach 3:

Model Pruning

- Pruning reduces the size and complexity of a neural network by **removing less important weights or neurons**.
- Performed L1 unstructured pruning permanently removing 20% of the least important weights across the model.

Metric	Seq 1 Benchmark	Seq 1 Pruned Model	Seq 4 Benchmark	Seq 4 Pruned Model
Translational error (%)	23.67	30.40	9.04	4.50
Rotational error (°/100m)	5.86	8.177	4.87	4.93
Absolute Trajectory Error (m)	101.70	54.35	6.00	1.13
RPE (m)	0.5415	0.4706	0.1414	0.1083
RPE (°)	0.3104	0.4022	0.1743	0.199



baseline	Checkpoint Size	Parameters (M)
	368.19 MB	30.6
pruned	Checkpoint Size	Parameters (M)
	122.74 MB	24.5

Pruning led to a significant reduction in ATE and RPE across sequences, with Sequence 4 showing a 50% drop in translational error and improved overall accuracy—indicating that pruning enhanced model performance rather than degrading it.



Conclusion

Optimization Approach 1: Refining the Pose Estimation Loss Function

- With the new loss function we observed an improvement in the pose accuracy, especially in rotation, which lead to more reliable and consistent predictions.

Optimization Approach 2: Weight Quantization

- Quantized models, despite being a fraction of the size, trade slightly higher pose errors for significantly improved trajectory smoothness and local consistency compared to the benchmark.

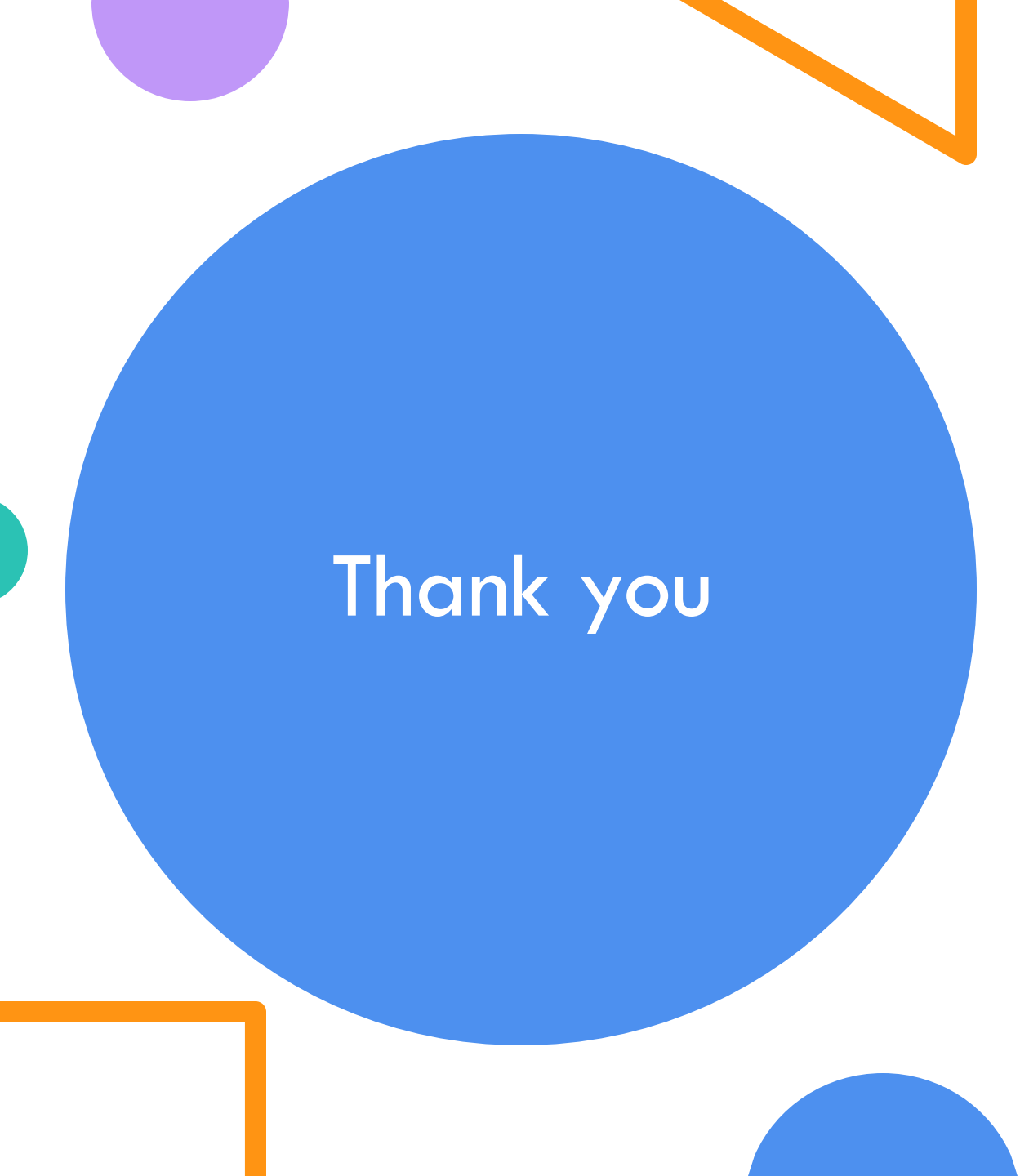
Optimization Approach 3: Model Pruning

- Pruning resulted in a significant reduction in ATE and RPE across sequences, while using only two-thirds of the parameters and one-third the size of the original model.

Next Steps -

For the next steps, we will focus on ensuring that all of our results are properly finalized. We will also begin working on the paper and keep the code ready for submission.





Thank you